

Calidad de Software

Ingeniería de Software 4

Luz Mery Rios Alzate | Ingeniería de Sistemas

Métricas de Calidad de Software
Basadas en ISO y CMMI

2

Universidad Católica de Oriente
Vigilada Mineducación

Analizar, comprender y aplicar métricas de calidad de software utilizando como referencia las normas ISO (especialmente ISO/IEC 25010) y los modelos de proceso como CMMI, con el fin de evaluar indicadores de un proyecto y tomar decisiones para la mejora continua.

Aspectos clave que puedes explorar:

- Concepto y tipos de métricas.
- Métricas del producto y del proceso.
- Relación con la calidad del software (ISO).
- Indicadores en proyectos de software.
- Métricas y niveles de madurez (CMMI).
- Interpretación y análisis de datos.
- Uso de métricas para la toma de decisiones.
- Mejora continua basada en evidencia.

Propósito de la actividad: Analizar un conjunto de indicadores de un proyecto de software utilizando métricas de calidad, con base en ISO y CMMI, para identificar fortalezas, debilidades y proponer acciones de mejora.



ISO/IEC 25010

- Calidad del producto
- Características de calidad
- Subcaracterísticas
- Medición y evaluación

CMMI

- Medición y análisis (MA)
- Gestión de procesos
- Uso de indicadores
- Mejora continua

"Lo que no se mide, no se puede mejorar"

Métricas de Calidad de Software (Actividad en Equipo o Individual)

Cada estudiante deberá desarrollar un mini-taller o caso de análisis que incluya:

A. INTRODUCCIÓN
- ¿Qué son las métricas de calidad de software?
- ¿Por qué son importantes en un proyecto?
- ¿Cómo se relacionan con ISO y CMMI?
- ¿Qué decisiones se pueden tomar con base en métricas?

B. MARCO DE REFERENCIA
- Calidad del software según ISO/IEC 25010.
- Procesos de medición en CMMI.
- Tipos de métricas: producto, proceso y proyecto.
- Ejemplos de indicadores comunes.

C. CASO DE ANÁLISIS
- Analizar un conjunto de datos de un proyecto (real o simulado).
- Calcular e interpretar indicadores de calidad (ej. densidad de defectos, cobertura de pruebas, tiempo de corrección).
- Identificar fortalezas y debilidades.

D. ANÁLISIS Y RESULTADOS
- Interpretar los resultados de las métricas.
- Comparar con valores de referencia o buenas prácticas.
- Identificar riesgos y oportunidades de mejora.

E. PROPUESTA DE MEJORA
- Proponer acciones para mejorar los indicadores de calidad.
- Justificar las acciones con base en ISO/CMMI.
- Establecer cómo se medirá el impacto.

F. CONCLUSIONES
- ¿Qué aprendiste del análisis de métricas?
- ¿Cómo ayudan las métricas a tomar mejores decisiones?
- ¿Qué otros indicadores serían relevantes?

G. PRODUCTO FINAL
- Documento o informe con el análisis.
- Tablas y gráficos de indicadores.
- Conclusiones y recomendaciones.
- Presentación de los resultados.

Incluir en la entrega:

- **2 REFERENCIAS BIBLIOGRÁFICAS** (1 sobre ISO y 1 sobre CMMI).
- **1 ARTÍCULO CIENTÍFICO O TRABAJO DE GRADO** relacionado con métricas de calidad.

Importante: las fuentes deben estar correctamente citadas (APA) y el análisis debe basarse en datos reales o en un caso simulado.

Deuda Técnica
en Proyectos de Software

2

UCO
Universidad Católica de Oriente
Vigilada Mineducación

ALTA CALIDAD

Analizar, comprender y proponer estrategias de gestión de la deuda técnica en un proyecto de software, reconociendo sus causas, consecuencias y su impacto en la calidad y sostenibilidad del sistema.



"Decisiones de hoy que pueden convertirse en costos mañana."

Propósito de la actividad

Comprender, analizar y argumentar el impacto de la deuda técnica en proyectos de software, identificando sus causas, tipos, consecuencias y alternativas para su gestión, a partir del análisis de un caso real o simulado.

1 A. INTRODUCCIÓN
Comprender el problema

- ¿Qué es la deuda técnica?
- ¿Por qué se utiliza el término "deuda"?
- ¿Cómo surge en un proyecto de software?
- ¿Toda deuda técnica es negativa?
- ¿Por qué puede ser una decisión válida?
- Diferencia entre deuda intencional y no intencional.

Pregunta para iniciar:
¿Entregar una funcionalidad rápidamente, sabiendo que después será necesario corregirla, puede ser una decisión válida de ingeniería?

2 B. MARCO DE REFERENCIA
Comprender el concepto

- Definición de deuda técnica.
- Origen del concepto.
- Características principales.
- Tipos de deuda técnica:
  - Deuda de código
  - Deuda de diseño
  - Deuda arquitectónica
  - Deuda de pruebas
  - Deuda de documentación
  - Deuda de infraestructura
  - Deuda de procesos
  - Dependencias
- Causas más comunes.
- Relación con la calidad del software.

3 C. CASO DE ANÁLISIS
Llevar el concepto a una situación real

- Seleccionar o recibir un caso de software (real o simulado).
- Descripción del sistema y su contexto.
- Problema o situación actual del sistema.
- Antecedentes y decisiones que generaron la deuda técnica.



4 D. ANÁLISIS DEL CASO
Comprender las consecuencias

- Identificar la deuda técnica existente.
- ¿Qué decisiones generaron la deuda?
- Consecuencias para el sistema, el equipo y el negocio.
- Riesgos asociados.
- Costos actuales y futuros.
- ¿Qué pasa si no se hace nada?

5 E. PROPUESTA DE GESTIÓN
Tomar decisiones profesionales

- ¿Qué harían con la deuda técnica?
- Acciones de mejora (refactorizar, rediseñar, mejorar, automatizar, documentar, etc.).
- Justificación de las decisiones.
- Recursos necesarios.
- Beneficios esperados.
- Riesgos y alternativas.
- ¿Qué deuda mantener, reducir o eliminar?

6 F. CONCLUSIONES
Reflexionar y argumentar

- ¿Qué aprendieron sobre deuda técnica?
- ¿Por qué es importante reconocerla durante el desarrollo?
- ¿La deuda técnica siempre debe eliminarse?
- ¿Cómo afecta la sostenibilidad del sistema?
- ¿Qué papel tiene el ingeniero de software en su gestión?

7 G. PRODUCTO FINAL
Exposición y entrega

- Presentación de 15 minutos + 5 minutos de preguntas.
- Todos los integrantes deben participar.
- Utilizar diagramas, tablas, ejemplos y evidencias.
- Documento o presentación que incluya:
  - Caso analizado
  - Identificación de deuda
  - Análisis de consecuencias
  - Propuesta de gestión
  - Conclusiones
  - Referencias.

Trabajo
Individual o en equipo de máximo 4 estudiantes.

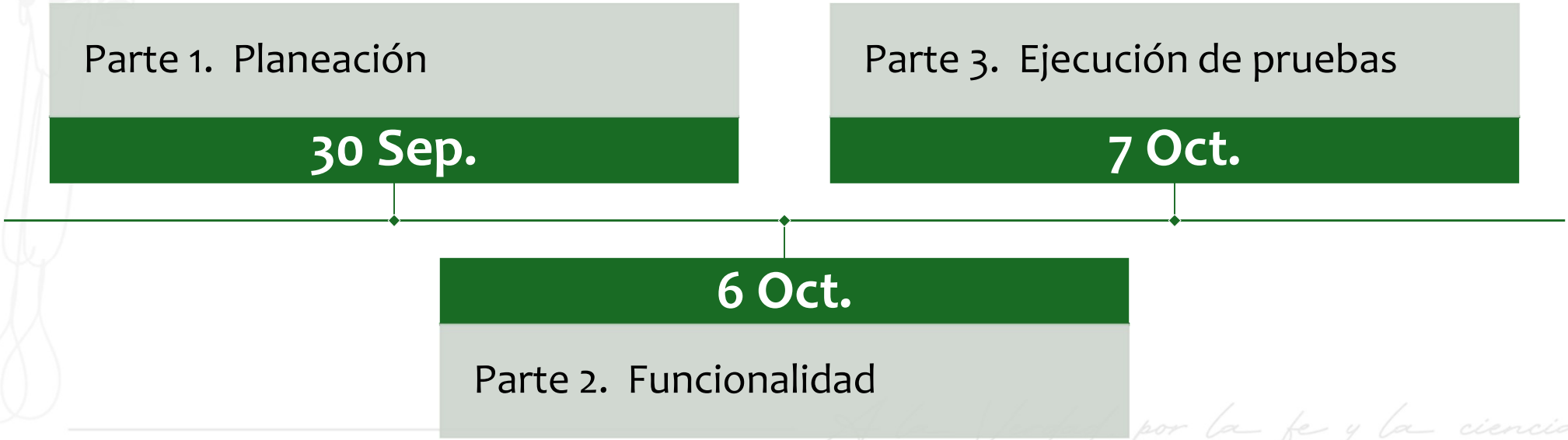
Tiempo de exposición
15 minutos + 5 minutos de preguntas.

Fuentes
Mínimo 2 referencias sobre deuda técnica y 1 artículo científico o trabajo académico. Citas en formato APA.

Criterios de evaluación
Comprensión del problema, análisis, argumentación, propuesta de gestión, calidad de la presentación y uso adecuado de fuentes.

¿Qué sigue?
Proyecto – Entrega de Implementación

Iniciar la implementación del proyecto mediante la planificación de las primeras actividades, la organización del equipo y la configuración del ambiente de desarrollo.



¿Qué sigue?

Proyecto – Entrega de Implementación

Parte 1. Planeación



1. Planner del proyecto

Entrega 6 octubre: Funcionalidad por estudiante terminada

Entrega 20 de octubre: Corrección de los errores y entrega final del proyecto.



2. Organización del proyecto

Definir roles.

Crear el repositorio.

Establecer la estructura inicial del proyecto.

Definir reglas básicas para el trabajo colaborativo.



3. Ambiente de desarrollo

Crear el proyecto.

Configurar las herramientas y dependencias.

Configurar la base de datos.

Verificar la ejecución del proyecto.

Crear README con las instrucciones para ejecutar el proyecto.



Entregable

Planner.

Enlace al repositorio.

Documento breve de organización del proyecto.

Evidencias de configuración y ejecución del ambiente.

README del proyecto.

A la Verdad, por la fe y la ciencia

¿Qué sigue?
Proyecto – Entrega de Implementación
Parte 2

Funcionalidad implementada

Código integrado

Commit(s) identificables.

Código asociado a la funcionalidad.

Integración con la rama principal o mecanismo definido por el equipo.

Evidencia de funcionamiento

Capturas de pantalla.

Video corto.

Evidencia dentro del sistema funcionando.

Pruebas integración

Mínimo 5 casos de prueba

Entregable

Repositorio actualizado

Planner actualizado

Ficha de cada funcionalidad y Casos de prueba.

¿Qué sigue?
Proyecto – Entrega de Implementación
Parte 2

Una funcionalidad se considera terminada cuando:

- ❑Está implementada.
- ❑Está integrada al proyecto.
- ❑Puede ejecutarse.
- ❑Cumple el comportamiento esperado.
- ❑Tiene validaciones básicas.
- ❑Tiene datos de prueba.
- ❑Tiene mínimo cinco casos de prueba definidos.
- ❑Está disponible en el repositorio.
- ❑Puede ser demostrada por el estudiante.

A la Verdad, por la fe y la ciencia

Referencias:

- Kaner, C., Bach, J., & Pettichord, B. (2002). Lessons Learned in Software Testing. Wiley. ISBN: 978-0471081128.
- Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley. ISBN: 978-1118031964.
- Jorgensen, P. C. (2013). Software Testing: A Craftsman's Approach (4th ed.). Auerbach Publications. ISBN: 978-1466560680.
- Crispin, L., & Gregory, J. (2009). Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley. ISBN: 978-0321534460.
- Graham, D., Black, R., & van Veenendaal, E. (2012). Foundations of Software Testing: ISTQB Certification (3rd ed.). Cengage Learning. ISBN: 978-1408044056.

Referencias:

- Whittaker, J. A. (2009). Exploratory Software Testing. Addison-Wesley. ISBN: 978-0321636416.
- Kinsbruner, E., Craske, A., & Feloney, S. (2019). Continuous Testing for DevOps Professionals. Apress. ISBN: 978-1484253505.
- Weinberg, G. M. (2008). Perfect Software and Other Illusions About Testing. Dorset House. ISBN: 978-0932633699.
- Whittaker, J. A. (2002). How to Break Software: A Practical Guide to Testing. Addison-Wesley. ISBN: 978-0201796193.
- Black, R. (2012). Advanced Software Testing - Vol. 1: Guide to the ISTQB Advanced Certification as an Advanced Test Analyst (2nd ed.). Rocky Nook. ISBN: 978-1937538497.

UCO
Universidad Católica de Oriente
Vigilada Mineducación

ALTA CALIDAD



¡Gracias!